

SensorAct: An IoT Middleware for Energy Management in Buildings

Journal:	<i>IEEE Internet of Things Journal</i>
Manuscript ID:	IoT-0333-2014
Manuscript Type:	Expanded Conference Paper
Date Submitted by the Author:	21-Sep-2014
Complete List of Authors:	Arjunan, Pandarasamy; Indraprastha Institute of Information Technology (IIIT), Delhi, MUC Saha, Manaswi; Indraprastha Institute of Information Technology (IIIT), Delhi, MUC Choi, Haksoo; University of California Los Angeles, Gulati, Manoj; Indraprastha Institute of Information Technology (IIIT), Delhi, MUC Singh, Amarjeet; Indraprastha Institute of Information Technology (IIIT), Delhi, Singh, Pushpendra; Indraprastha Institute of Information Technology (IIIT), Delhi, Srivastava, Mani; University of California Los Angeles Los Angeles,
Keywords:	Sensor Device Middleware < Sub-Area 1: Sensors and Devices for IoT, Sensor System Integration < Sub-Area 1: Sensors and Devices for IoT, Application Platform < Sub-Area 3: Services, Applications, and Other Topics for IoT, Cyber-Physical Systems < Sub-Area 3: Services, Applications, and Other Topics for IoT, Data Management and Analytics < Sub-Area 3: Services, Applications, and Other Topics for IoT, Smart Environment < Sub-Area 3: Services, Applications, and Other Topics for IoT, Test-bed and Trials < Sub-Area 3: Services, Applications, and Other Topics for IoT

SensorAct: An IoT Middleware for Energy Management in Buildings

Pandarasamy Arjunan*, Manaswi Saha*, Haksoo Choi[†], Manoj Gulati*,
Amarjeet Singh*, Pushpendra Singh*, Mani B. Srivastava[†]

*Indraprastha Institute of Information Technology, Delhi, India
{pandarasamy,manaswis,manojg,amarjeet,psingh}@iiitd.ac.in

[†]University of California Los Angeles, United States
haksoo@cs.ucla.edu, mbs@ucla.edu

Abstract—Buildings, with their different subsystems interacting with diverse occupants, constitute a complex Cyber-Physical-Human infrastructure. Monitoring and controlling this complex ecosystem is essential both for efficient and optimized operations of building subsystems and for influencing the occupant behavior. A critical enabling technology in this case is a middleware system for buildings that can provide support for deriving rich inferences by fusing and analyzing intentionally acquired or opportunistically available data from diverse embedded sensors, human feedback, and existing building subsystems. This paper presents *SensorAct*, a decentralized IoT middleware system architecture for energy management applications in buildings. In addition to providing support for managing and integrating heterogeneous sensing and actuation systems in buildings, *SensorAct* provides two emerging features: 1) a *scripting framework* for application developers to extend and automate the energy management functions of the buildings, and 2) a *rule-based sensor data and control sharing* mechanism for fine-grained sharing for building owners. We describe the detailed system architecture and design, and provide proof of concept through multiple third party applications built using *SensorAct* API and deployment in diverse settings across India and United States. *SensorAct* is released in open source for community use.

Index Terms—Energy monitoring, Internet of Things

I. INTRODUCTION

Buildings account for a significant proportion of overall energy use in both the developing (e.g., 47% of total energy in India [1]) and the developed (e.g., 41% in US [2]) countries. Energy consumption in buildings is spread across diverse devices, appliances and control subsystems including lighting, Heating Ventilation and Air Conditioning (HVAC), security and access control. Interdependence of these sub-systems, together with occupancy of buildings by diverse users make buildings a complex *Cyber-Physical-Human* system. Detailed understanding of different subsystems within buildings, together with their interactions with diverse occupants, is critical for developing solutions that optimize overall operations of buildings and scale across diverse usage patterns.

Commercial buildings often employ Building Management Systems (BMS) for monitoring and controlling of its sub-systems. While striving to achieve an optimal energy efficient control, these BMSes restrict the management to a central facility department. To support interoperability, many of these BMSes support Building Automation Controller Network (BACnet)¹. However, external third party application

development using BACnet interface is complex so it restricts widespread development of such applications [3]. Such third party applications, if facilitated, can further support integration of different building sub-systems for improved operations. For example, information from RFID based access control system can be used to infer the occupancy and accordingly control the HVAC.

At the other end of the spectrum, for residential buildings, several home monitoring and automation applications have emerged recently. These applications typically involve monitoring ambient parameters (e.g., motion and temperature) across home together with smart metering solutions for detailed logging of whole-home electricity, gas, and water consumption. Observations from such monitoring systems are often used to control various home appliances, e.g., thermostats, water heating, and lighting. Correspondingly, several cloud-based Internet of Things (IoT) and Machine to Machine (M2M) communication platforms have been proposed recently². These platforms allow users to connect their devices, including both sensors for monitoring and actuators for controlling, to a centralized service, and build their own applications using the observed parameters and available actuators.

While these existing cloud-based IoT systems are a step forward towards developing new applications for monitoring and controlling the energy usage in buildings, their centralized architecture results in several limitations as follows:

- 1) **Data privacy:** Detailed monitoring in both commercial and residential buildings results in large volumes of data that can be interpreted to infer several personal attributes [4], e.g., home occupancy or wake-up times. Privacy concerns emerging from such data, hinders some users to publish their data to a centralized cloud service [5].
- 2) **Intermittent network connectivity:** Intermittent network connectivity issues, especially in the context of developing countries, are de-motivating the reliance on always connected devices to cloud-based IoT platforms², specifically for the buildings domain.
- 3) **Real-time control:** Data collection and control actions occurring in a cloud are further associated with latency issues arising out of network delays and high usage sce-

¹<http://www.bacnet.org>

²<http://xively.com>, <http://www.nimbts.com>, <http://open.sen.se>,
<http://mango.serotoninsoftware.com>

narios. Some operations within buildings, e.g., turning on the sprinkler in case of fire, need real time control that may be difficult to obtain through cloud IoT services.

Motivated by aforementioned limitations, in this paper we present *SensorAct* - an IoT middleware architecture specifically designed for detailed monitoring and control in buildings. Beyond connecting devices and thereby monitoring the built spaces, *SensorAct* provides emerging capabilities including:

- 1) *Virtual Personal Device Servers (VPDS)* (See Section II-B) for local-hosting of middleware within the buildings to alleviate data privacy, control security and intermittent network connectivity problems.
- 2) Decentralized and distributed management of building resources involving diverse devices and different stakeholders, including the occupants, at scale. Such decentralized architecture (keeping the middleware closer to the devices) facilitates real time control required for energy management applications.
- 3) Fine-grained selective sharing of sensor data and control with users at the global scale to alleviate control security concerns while providing building-wise local storage for protecting the data privacy.
- 4) *Scripting framework* (See Section III), simple programming abstractions for extending the system features and developing energy managing applications involving sensing and control, analysis, alerts, and notifications in rich form to support diverse usage scenarios.

Together with supporting diverse usage scenarios, *SensorAct* accommodates a rich ecosystem of existing and new monitoring and controlling systems. Correspondingly, the key contributions of this paper are:

- A working middleware system, released in open source, that can run on heterogeneous platforms, allowing participants to collect data from a variety of sensors and perform actions thereof.
- A *scripting framework* to extend the middleware system functionality with customized application logic.
- A rule-based *fine-grained access control* mechanism enables sharing of sensor data and actuation control with other participants.
- Detailed evaluation of the system through extensive deployments involving different usage scenarios and energy monitoring applications.

A preliminary design of *SensorAct* was presented earlier in 2012 Workshop on Embedded Systems for Energy Efficiency in Buildings (Buildsys) [6]. Significant system development leading to multiple real world deployments and extensive evaluation of the scripting framework for various energy monitoring applications, are all new inclusions for this paper.

II. SENSORACT ARCHITECTURE

The primary objective is designing an open source IoT middleware system for energy management in buildings. *SensorAct* adopts a *tiered architecture* connecting the building subsystems with its occupants as shown in Figure 1. The main layers of *SensorAct* are *Virtual Personal Device Servers (VPDS)* and *Brokers*, which interact with *Devices* that monitor

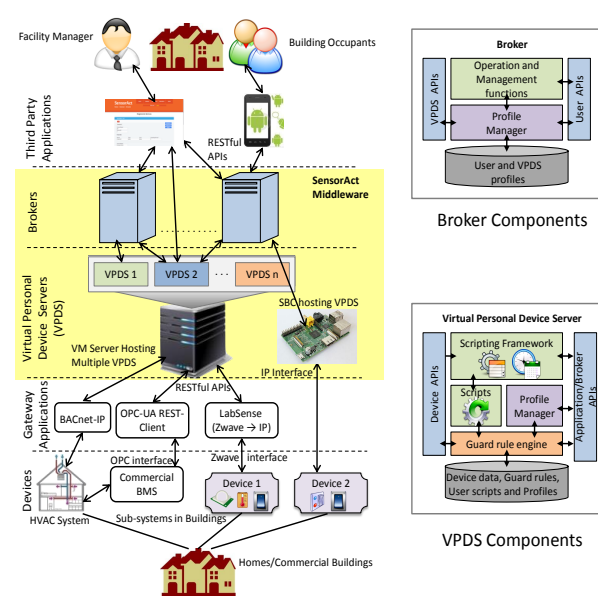


Fig. 1: Tiered architecture of *SensorAct* and its various sub-system components

and control different building spaces at the lowest layer, and third-party applications and users at the highest layer.

A. Devices and Gateways

Existing building subsystems contain numerous sensing and actuation points connected using diverse protocols such as BACnet and Modbus. *SensorAct* maps the existing and new sensing and actuation points as *Devices* by eliminating the underlying complex naming conventions used in the legacy building subsystems. A device in *SensorAct* consists of a collection of sensors (monitoring the ambient environment or the state of a building subsystem) and actuators (that allow for changing the state of a building subsystem). Each sensor, in turn, may consist of a collection of channels which measure a particular building phenomenon. *SensorAct* uses an intuitive and hierarchical naming scheme for managing and identifying the devices and their sensors/actuators, channels and readings uniquely. As an example, a single reading from a smart energy meter connected with the main panel is identified as *OwnerName/MainPanel/EnergyMeter1/power/ < timestamp > / < value >*.

Devices are assumed to either communicate directly or through a gateway (e.g., LabSense, as discussed in Section IV-A) that can bridge the sensing interface with *SensorAct* using its RESTful API. Each device is associated with a device profile that contains all meta information. A building owner can manage multiple devices owned by him through the *profile manager* facilitating the creation of device profiles. It consists of a set of key-valued paired attributes such as its name, IP address, a collection of sensor and/or actuator profiles, number of channels, data types and units, location and placement.

B. Virtual Personal Device Server (VPDS)

VPDS is the primary core component of *SensorAct* middleware. As a package, VPDS contains (a) *Data Archiver* for storing and retrieving time-series sensor data, (b) *Scripting Framework* for executing the custom building applications, (c) *Guard Rule Engine* for access control, and (d) *Profile Manager*

and APIs for devices, applications, and brokers to interact with the VPDS, as shown in Figure 1. VPDS Owner (VO) may allow other users to have controlled access to the devices registered with the VPDS. These devices monitor and control the buildings owned by VO. Each VPDS has an associated auto-generated owner's key which is used while registering the VPDS with Broker for data and control sharing.

The VPDS abstraction not only permits flexible provisioning of hardware resources but also allows diverse deployment scenarios including a VPDS hosted on a local machine for high frequency sensors, low dependence on Internet connectivity, high-security and low-latency sensing and control. Such multiple VPDS instances are coordinated through Brokers in the higher tier. VPDS-Broker architecture further enables global access through distributed registry of users.

C. Broker

In *SensorAct*, a trusted broker contains a registry of users, a registry of VPDSes, and it acts as a mediator to assist client applications establish a connection with multiple VPDSes. A VO needs to register her VPDS on one of the brokers to share data and control with other users registered on the corresponding broker. For data communication to scale across multiple VPDSes and users managed by a single broker, direct communication over a secured channel is provided between users and VPDSes.

D. Applications

Programming abstractions in *SensorAct*, using RESTful APIs, allow easy development of third party applications that provide a user with controlled access to sensor data and actuators. A *SensorAct* user may have two roles: i) An owner of her own devices and correspondingly associated VPDS ii) A user with data and control access as per the privileges assigned by the owner of other VPDS. A single user could be the owner of her own VPDS and user for some other VPDS. As an owner, a user may grant controlled access to other users thus letting them access sensor data from her devices or to even control them in a constrained manner. Third party applications may provide users with more convenient interface to the underlying functionality of VPDS and broker. We discuss three such third party applications in Section IV.

E. Guard Rule Engine for Selective Sharing

Guard Rule Engine in *SensorAct* is designed to support *selective sharing* of sensor data and actuation control with other users governed by the corresponding owner. All access requests for actuators or sensor data are governed by Guard Rule Engine. Tight control on the access is maintained through owner-defined *guard rules* which are policies for restricting access to the data and control of the devices configured for buildings. Guard Rule Engine enables *fine grained access control* by allowing the owner to define rules based on user, group, time, location, sensor data, and actuators. Every device registered with the VPDS has an associated set of guard rules, created by corresponding VO, for facilitating external (or shared) access.

The specification of the guard rules and its features such as macros, templates, and built-in functions can be found in [6]. Figure 2 shows an example of a guard rule and a macro.

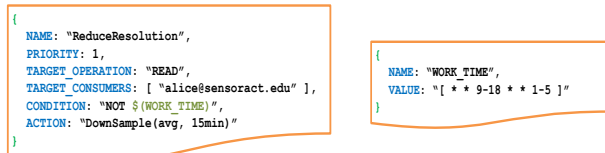


Fig. 2: An example of a guard rule and a macro.

This guard rule enforces a policy to share data during non-working hours only, which is defined by the macro using *Cron* time expression, and reduces the data access resolution to 15 minutes. The guard rule itself does not necessarily contain references to specific users, sensors, or actuators, but it can be associated to them later by its owner. This *lazy-association* allows users to reuse same rules for different users, multiple devices, or groups of devices. More details on sharing process through Guard Rule Engine can be also found in [6].

III. SCRIPTING FRAMEWORK: PROVIDING SIMPLIFIED AUTOMATION

The *Scripting Framework* in *SensorAct* provides an application execution environment within the middleware for high-level scripting languages in a sandbox. Unlike other IoT platforms, where in external applications read sensor data and perform control actions outside the middleware, this enables building owners to inject their custom application logic written in a high-level scripting language, termed as *tasklets*, into the middleware to perform sophisticated energy management and control operations. These tasklets can be scheduled to read and process live sensor data streams. The proposed *Scripting Framework* also provides a set of read functions to access sensor data streams and write functions to control the actuators. These read/write primitives can then be used to develop complex (one-shot or persistent) control actions providing *rich support for automation* including sensor data processing, data fusion, actuator control, and notifications. Figure 3 illustrates the workflow of the proposed scripting framework.

A. Tasklet Workflow

A *tasklet* in the scripting framework is a piece of lightweight non-blocking script that performs a particular set of operations. As shown in Figure 3, it consists of *tasklet description* and *tasklet script*. While the *tasklet description* contains meta information about the tasklet, such as resources (sensors and actuators identifier) to be used, parameters, scheduling and triggering conditions, *tasklet script* contains the application logic in a high-level scripting language. More details about tasklet description can be found in [6]. A *tasklet scheduler* receives the tasklet execution requests, submitted using the corresponding tasklet management API provided by the VPDS, and it is responsible for scheduling, managing, and controlling the tasklet throughout its life-cycle. Once a tasklet is scheduled for execution, the *tasklet scheduler* first classifies and schedules the tasklet, based upon the identifiers used in the *When* primitive of the description, as one of the following categories:

One-shot: One-time and immediate execution of a tasklet script. For example, querying the current status of a sensor or instantaneous switching of an actuator.

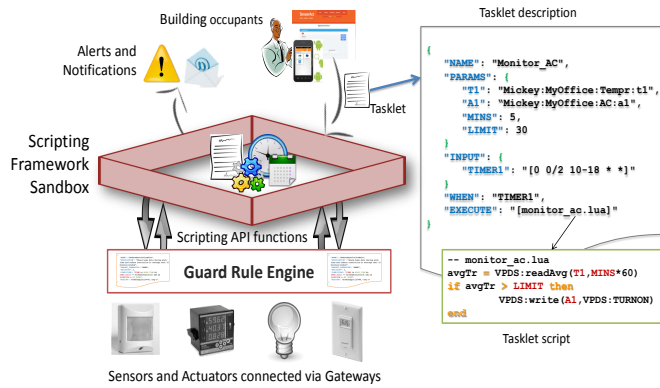


Fig. 3: Scripting framework workflow with an example Tasklet

Periodic: Executing a tasklet script whenever timer elapses to perform periodic operations. For example, switch on my office air-conditioner at 9AM only on week-days or email me the electricity usage summary every day at 8PM.

Event driven: Executing a tasklet script whenever a change in the value of sensors or actuators is observed. For example, switching off lights when a window is opened.

Tasklet manager provides an isolated execution environment for each tasklet and restricts any interaction among them for sensitive building control applications. By default, a tasklet inherits the privileges of a user who submitted it. All read and write operations on sensors and actuators performed from tasklets go through Guard Rule Engine. Correspondingly, tasklets can only perform operations allowed for a user invoking them, thus protecting against unauthorized access.

B. Tasklet API Functions

The scripting framework in *SensorAct* provides a set of primitive API functions as shown in Table I, in order to perform various runtime operations. Tasklet scripts can invoke these low-level API functions and it enables developers to create and schedule custom building control and automation applications. While expert users can write complex tasklet scripts on their own, novice users can use an interactive web interface to create simple automation applications. We also plan to provide *tasklet templates* for commonly used applications, so that users can easily create tasklets by filling in only the required parameters.

The proposed tasklet framework provides a platform to implement and schedule several energy management tasks within the middleware system. Examples are 1) inferring high-level rich occupant-specific context information, such as occupancy and usage patterns, by fusing several raw sensor and actuator values; 2) automation of the routine building control activities performed by the occupants in their day-to-day life, e.g., pre-heating or pre-cooling the workspaces in advance; 3) custom creation of *coordinated appliances* based upon detected building events, e.g., switching off a meeting room may result in switching a group of devices together or in a particular sequence; and 4) development of an automated alert or notification system for monitoring and management of

building premises, e.g., alerting the facility management team in case of any abnormal energy consumption. In Section V, we present a list of energy management applications created using the proposed tasklet framework.

IV. IMPLEMENTATION

A. Gateways and Devices

Gateways interconnect existing and new sensors and actuators in the buildings with VPDS. In the current implementation, three gateways are supported: 1) LabSense¹⁰ for interfacing Z-Wave based ambient sensors (temperature, light intensity, motion, and door contact status) in existing Home Automation Systems, Modbus based smart energy meters, SNMP based Raritan³ power distribution unit, and Modbus based Veris and Eaton⁴ energy meters, 2) sMAP [7] to communicate with several commercial energy meters and HVAC systems using different protocols such as Modbus and BAC-Net, and 3) A custom built Wi-Fi based Flyport⁵ module to interface ambient sensors and actuation relays. These gateways push sensor readings in an uniform format, as described in [6], to VPDS using RESTful APIs, shown in Table II, and execute the actuation commands received from the building control applications.

B. VPDS and Broker

Various VPDS and Broker components, as explained in Figure 1, were implemented in Java using open source tools and released the code for community use⁶. *SensorAct* exposes a rich set of RESTful APIs for most of its functionalities. Such open APIs enable easy integration with other systems and allow developers to write custom third party (stand-alone, web, or mobile based) applications for extending the system features, e.g., visualization, scripting, access control, and sharing. Table II lists the APIs currently implemented for various components in VPDS and Broker.

The scripting framework in *SensorAct* uses Quartz⁷ library to schedule and execute *tasklets*. The current implementation supports Lua⁸ and Jython to write tasklet scripts. Lua was chosen because it is a light-weight, compact, and easy-to-learn scripting language which also has been widely used to program home automation systems. Jython, an implementation of the Python scripting language in Java, provides rich support for data processing. Further, an VPDS instance can be hosted on multifarious devices including single board computers such as Raspberry Pi.

C. Applications

Three third-party applications were implemented using the VPDS and Broker APIs and they are explained below.

Web portal: It is a configurable stand-alone web application that interacts with a broker and registered VPDSes using the *SensorAct* APIs. Both VPDS owner and other users can use this application to manage devices, guard rules, and

³<http://www.raritan.com/>

⁴<http://www.veris.com>, <http://www.eaton.com>

⁵<http://www.openpicus.com>

⁶<http://github.com/iiitd-ucla-pc3>

⁷<http://quartz-scheduler.org>

⁸<http://www.lua.org>

```
map read(DeviceId, duration(in seconds))
map read(DeviceId, startTime, endTime)
number read(DeviceId, duration, [sum|count|min|max|mean] )
number read(DeviceId, startTime, endTime, [sum|count|min|max|mean] )
string plot(DeviceId, duration)
boolean write(DeviceId, status=ON|OFF|value)
boolean email(to, subject, message [,plot])
boolean sms(to, message)
```

TABLE I: Primitive API functions in Scripting framework available to use in Tasklet Scripts

Component	VPDS API endpoints
User	/user/{register list}
Device	/device/{add delete get list}
	/device/template/{add delete get list}
Guardrule	/guardrule/{add delete get list}
	/guardrule/association{add delete}
Tasklet	/tasklet/{add delete get list}
	/tasklet/{execute cancel status}
Data	/data/{upload/wavesegment query}
Share	/device/share
Component	Broker API endpoints
User	/user/{register login list}
VPDS	/vpds/{register get list}
Device	/device/{search share}
	/device/{user owner}/shared

TABLE II: Supported SensorAct APIs for different system components

tasklets based upon granted privileges. An integrated plotting application also enables users to visualize their sensor data.

Time and presence based actuation: It is an intuitive web interface, extended from the web portal application, which allows users to actuate their devices remotely based on time and presence, e.g., pre-heating/cooling a workspace. The user interface makes use of tasklets to allow users to switch their appliance “now”, “once” at a specific time, or “periodically” at a regular time interval. Guard rules are used to restrict users to actuate the devices in their own spaces only. This system is currently under deployment in a commercial building for lighting control.

Mobile application: An Android application was developed whereby users can specify their Broker credentials and correspondingly manage the devices owned or shared with them.

Further, in order to simplify the *SensorAct* system installation for a building, all VPDS, broker and user interface components of *SensorAct* were packaged into a Virtual Machine image. Detailed installation instruction manual were documented and the usage of the system was evaluated using an user study (See Section V-C).

V. EVALUATION

The proposed *SensorAct* architecture is evaluated based on multiple real-world deployments for supporting different usage scenarios. Particularly, the utility of the proposed tasklet framework is shown for various energy monitoring and alerting applications. Further, an user study was performed to evaluate the different aspects of deployment of *SensorAct* in a student dormitory building.

SensorAct was deployed in four different settings as illustrated in Figure 4: (1) Campus wide energy monitoring at IIIT-Delhi, India, (2) Student dormitory deployment at IIIT-Delhi, India, (3) Research lab in UCLA, USA, and (4) Research wing at IIIT-Delhi. Each deployment was done with different requirements, devices, gateways, users, and scales as shown

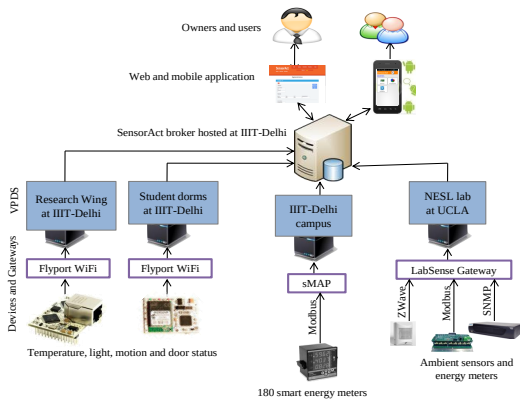


Fig. 4: Various deployment scenarios of *SensorAct* architecture. Four VPDSes deployed across different locations were connected to a common broker hosted in IIIT-Delhi

in Table III. Separate VPDS instances were used for each deployment, and they were registered with a common broker, hosted at IIIT Delhi, for sharing sensor data and control with users across different buildings. In each deployment, the corresponding owner or tenant of the building managed the VPDS and granted privileges to other occupants who all were registered themselves with the common broker, if required.

A. Campus Wide Electricity Monitoring at IIIT-Delhi

One of the largest deployments of *SensorAct* involved monitoring the electricity usage of all the buildings in IIIT-Delhi campus. IIIT-Delhi campus was newly constructed two years ago in a space of 25 acres. It consists of five buildings: academic, facilities, faculty apartments (30 flats), mess and hostel (400 dorm rooms) buildings. All these buildings are equipped with a commercial BMS system for managing the various building operations, under the administration of a facility manager (FM). In addition to the commercial BMS system, all the buildings (each floor and flats) were instrumented with over 180 smart meters measuring various electrical parameters. A sMAP based archiver was used for collecting meter readings at every 30 seconds. Existing BMS subsystems, such as HVAC, were also interfaced with the sMAP archiver using BACnet and Modbus bridge. A separate sMAP to *SensorAct* gateway was implemented for uploading all real-time measurements to *SensorAct*. As per our short discussion with the FM, he wanted us to setup three applications for managing the energy usage of the campus. Those energy management applications were created using the proposed tasklet framework and they are being used by the FM.

1) *Abnormal street light usage detection:* IIIT-Delhi campus contains pathways around the campus for about 2 kilometers. There are about 135 street lights installed in the path way. These street lights are manually switched on in the

Deployment	Research Wing	Student Dorms	IIITD Campus	NESL, UCLA
Purpose	Occupancy sensing and data sharing	Occupancy sensing and data sharing	Energy monitoring and alerts	Occupancy sensing and energy monitoring
Scale	Single building	21 student dorms	6 buildings	Research lab
Platform	Virtual machine	Laptop and PCs	Virtual machine	Virtual machine and MiniITX
Sensors	Ambient sensors (14)	Ambient sensors (21)	Electricity meters(180)	Ambient sensors and electricity meters(3)
Sampling rate	1 second	1 second	30 seconds	2 seconds
Gateway & protocols	FlyPort, WiFi	FlyPort, WiFi	sMAP, Modbus	LabSense, ZWave, Modbus
Duration	2 months	1 month	4 months	8 months
Users	20	21	2	15

TABLE III: Different deployment scenarios of *SensorAct*. Ambient sensors include Temperature, light intensity, motion and door contact status

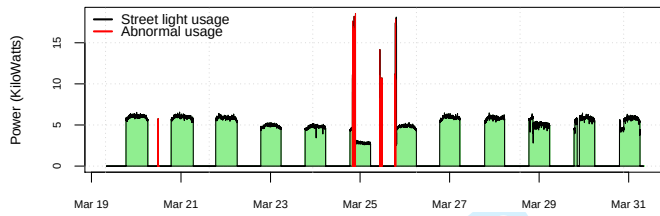


Fig. 5: Street light electricity usage pattern (6:30pm to 6:00am every day) for 12 days and abnormal usage events.

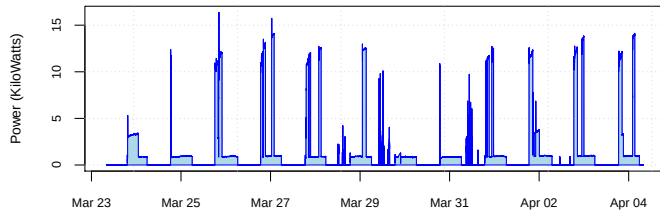


Fig. 6: Electricity usage patterns of sports area lights for 12 days. Few street lights around the sports area consuming about 0.9 kilo-watts of power are used from 6pm to 6am every day. Each spike in this plot corresponds to the usage of flood lights in the sports area.

evening and switched off in the morning by an operator. They consume over 6 kilo-watts of power. From the street light meter readings, as shown in Figure 5, we observed that there were some suspicious electricity usage events during day time and occasional events during night time as well. Hence, to monitor such abnormal electricity usage events by street lights, two tasklets, one for day time and another for night time were setup. They are computing average electricity consumption of the street lights every five minutes. If the average consumption is above a threshold (derived based on our observation), these tasklets send an email and SMS to the FM for taking necessary action. Over the course of past one month of this setup, these tasklets detected two such abnormal electricity usage events and notified the FM. The FM asked the facility support team to check the street lights and its power meters. FM also suggested that such period monitoring tasklets for street lights are essential, particularly in India, as electricity theft is not uncommon.

2) *Sports area usage summary*: IIIT-Delhi campus has a sports area that consists of a basket ball court, a foot ball court and a common play ground. The entire sports area is equipped with several flood lights and they consume over 10 kilo-watts of power when they are in use. At present, students are advised to turn on and off these lights whenever they want

to play during night time. For accountability and to make policies for the sports area usage, FM wanted to know how many hours these sports area was being used every day and the corresponding energy usage.

Since the sports area lights were connected through a separate smart meter, we monitored the electricity usage for a month to know the baseline usage. Figure 6 illustrates power consumption of the sports area lights (all the peaks) along with some other constant load for 12 days. Based on our observation, a periodic tasklet was created and it was scheduled to run at 8 A.M. everyday. The tasklet reads the smart meters readings for the previous night and filtered out the readings only for the sports area usage based on a known threshold value. The tasklet is also configured to send a summary email to the FM about how many hours the sports area was used and total energy consumption.

3) *Critical energy usage alert*: IIIT-Delhi campus receives two power lines from the grid, one for commercial and another for residential usage. While the residential power line is being used for faculty apartments and student's hostels, commercial power line is being used by the rest of loads in the campus. External commercial load from the grid is stepped down using two transformers, one for supplying high-voltage loads such as HVAC, and another one for connecting commercial usage such as lighting and IT devices. As shown in the Figure 7, commercial energy usage for a typical working day is over 3,000 units and it varies based on many other factors. In order to monitor the overall energy usage, FM wanted to develop an application that can alert him when the total energy consumption of the current day exceeds the previous day.

A periodic tasklet was created for monitoring the campus wide energy usage in real time. The tasklet was scheduled to be run at every five minutes. The tasklet script was configured to perform the following tasks in each run: 1) it reads the previous day and current day energy usage from the smart meters which are connected with the corresponding transformers, and 2) it compares them and sends an email and SMS to the FM if the current day energy usage exceeds the previous day consumption, 3) Since this is a periodic tasklet and to avoid continuously sending the notification on successive alerts, it was configured to send alerts two times maximum in a day.

B. Research Lab at UCLA

In this deployment, *SensorAct* was deployed for a 1,200 square feet university research lab, occupied by 12 graduate

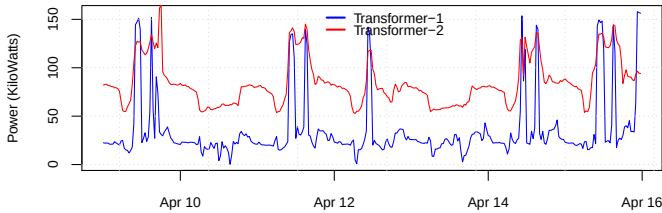


Fig. 7: Campus-wide total commercial electricity usage (from two transformers) for 1 week. The spikes in transformer 1 corresponds to the electricity consumption of HVAC systems, that consumes over 100 kilo-watts.

students, in UCLA for occupancy and electricity monitoring applications. It was instrumented with several sensors including 1) high frequency multi channel Raritan, Eaton, and Veris power meters measuring several electrical parameters of the lab servers, devices, and power outlets, and 2) Z-Wave based Aeon Labs door sensors and HomeSeer multi-sensor (measuring motion, temperature, and light intensity level) was interfaced with Mi Casa Verde Vera⁹

LabSense¹⁰ gateway was used to pull data from all of these sensors and push it to *SensorAct*. FireSense¹¹ system was also used to monitor the real time network traffic of the lab and the network event logs were pushed to *SensorAct* in real time. Access for sensor data and creating tasklets was shared with one of the graduate student at IIIT-Delhi, who created the computed sensor for presence detection (discussed in Section IV) and performed occupancy based experiments.

Occupancy and energy usage summary: Based on this setup, a periodic tasklet was created for monitoring the occupancy and electricity usage of the lab. The tasklet was scheduled to be run at 12 A.M. midnight every day to send a summary report to the lab members. The tasklet script was primarily doing three functions: 1) it decides whether some one is entering or exiting the lab by fusing the door status (open/close) with motion sensor events, 2) it compares the entry and exit events of the two doors and decides the first entry and last exit time of the lab, 3) it aggregates the total power consumption of a day and creates a plot of power consumption at 5 minutes, and 4) finally, it sends a summary email saying how many hours lab was opened on a particular day and how was the total power consumption during that time. This energy usage summary report were useful for giving insights about the usage patterns of the lab and for increasing energy usage awareness of the occupants. Further, the inferred occupancy information can be used to pre-heat/cool the lab.

C. Student Dormitory Deployment at IIIT-Delhi

In this deployment, in order to validate the utility and easy deployment of *SensorAct*, 21 student groups (each with 2 students at IIIT-Delhi) were engaged. They deployed Flyport based Wi-Fi nodes in student dormitory rooms, collecting motion, temperature, and window status information every second, by following the installation manual. One of the faculty coordinator hosted a central VPDS on one of the servers and registered it with the broker at IIIT-Delhi. Students

were then asked to register themselves and room occupants (for the case when they are deploying in someone else's room) on the broker. Faculty coordinator first shared privileges with the engaged student groups and allowed them to create devices in his VPDS though they were not allowed to see data from the devices added by them. This privilege was revoked after two days, and the added devices were shared with the corresponding room occupants to let them decide whom they want to share data. While, the study mandated each student to collect only 2 days worth of data from their individual deployment, a total of 100 days worth of data, together from all the groups was reported by the students.

A survey was conducted at the end of the deployment to get feedback from these students on different aspects of using the *SensorAct* system. Overall 17 student groups responded to the survey at the end of the study. Among them 16 student groups had no prior experience with uploading data to a server or cloud system such as *SensorAct*. More than 80% of the respondents mentioned that *SensorAct* installation and configuration on their individual laptops, with different OS, was easy. Approximately 45% of respondents gave their preference for local hosting of *SensorAct* VPDS instead of cloud, due to privacy reasons, if they were to deploy *SensorAct* for monitoring and control in their homes. Students used *SensorAct* for basic data collection and visualization while the occupants used sharing to provide access of their devices to their friends. 64% of them found *SensorAct* to be a good and usable system. More than 90% of responses rated *SensorAct* documentation to be detailed enough with 73% of them asserting that, with the current level of documentation, a new person can setup *SensorAct* system without any help. More than 90% of the responses were positive about the usability of the browser application that was used as a front end for the study.

VI. RELATED WORK

A. Cloud IoT Services

Several public cloud-centric IoT platforms exist today for collecting, archiving and visualizing the real time sensory data such as Xively, Nimbits, and Sen.se². While these services provide rich support for data aggregation and visualization of sensory data collected from diverse devices, they provide inadequate or limited support for IoT applications specifically pertaining to the buildings domain: 1) They provide limited capabilities for sharing the collected sensory data as they follow “all-or-nothing” model based only on user identity. Since the sensory data collected from buildings carry several forms of occupancy and usage patterns about the buildings, devices and occupants [8], controlled sharing is required to protect the sensitive sensory data collected from buildings. 2) Limited or no support for controlling the devices and automating their operations. A few services such as Sen.Se provide remote actuation support but they handle simple use cases wherein control is manual or is based on events, e.g., whenever motion is detected, switch on/off an appliance. *SensorAct* architecture allows for easy support of complex energy management applications using the scripting framework. In addition to the features provided by these systems, *SensorAct* supports rule-based selective sharing model for sensory data.

⁹<http://micasaverde.com>, <http://www.homeseer.com>

¹⁰<https://github.com/nesl/LabSense>

¹¹<https://github.com/nesl/FireSense>

B. Building Management Systems

Several commercial building management systems (BMS)¹² and home automation systems⁹ are currently available in the market for monitoring, controlling, and automating various building subsystems and operations. Typically, they comprise of several isolated subsystems each performing a particular task such as fire alarms, security and access control and HVAC. While they include a large number of sensing points spread across the building, data from these sensors are usually inaccessible to building occupants as these systems are normally controlled and managed by a central facility department. While many commercial buildings already have some form of BMS in existence, *SensorAct* can be used to augment them to develop novel occupant-centric applications such as personalized control of workspaces. Gateway applications can be easily developed to interface such M2M applications with *SensorAct* (as shown in Figure 1). Further, higher costs typically associated with such BMSs, prohibit their usage across small deployments such as in residential homes. Home automation systems, try to fill in the gap providing comfort to the occupants, using local storage and several automation scripts. However, they provide limited support for fine-grained data and control sharing across multiple homes, supported extensively in *SensorAct*.

C. Research Systems and Architectures

Several research systems pertaining to connecting and sharing sensors at Internet scale, such as SenseWeb [9], SensorWeb [10], GSN [11], and WattDepot [12] have been developed and deployed in the recent past. However, these systems are limited primarily to sensory data aggregation and visualization and provide minimal sharing capabilities. Some research software systems have been proposed in the literature that provides an abstraction over diverse devices and enables uniform interfaces to access them [13], [14]. For example, HomeOS [14] addresses the interoperability and usability issues by providing a PC like abstraction over the networked devices as peripherals for both users and developers. Sensor Andrew [15] shares some common design goals with *SensorAct*. It focuses on a large scale data aggregation from diverse sensors and event-based control for building operations. However, the data and control sharing mechanism is at coarse-grained level, based only on user identity. BuildingDepot [16] focuses on managing network of buildings by isolating the data, users, and privilege management from each other. On similar lines, Building Operating System Services (BOSS) and Building Application Stack [17] enable writing portable and fault-tolerant applications on top of diverse physical resources present in buildings. Specific to residential buildings, VHome [5] provides an isolated application execution environment for data analysis. Though VHome shares several design goals with *SensorAct*, it provides an application runtime to execute Cloud Based Application, focusing only on energy data analytics. Further, access control mechanism in VHome supports only time and location based energy data sharing, whereas *SensorAct* supports fine-grained sharing.

VII. CONCLUSIONS

In this paper, we presented the design and implementation of *SensorAct* - an open source IoT middleware system for energy management in buildings. *SensorAct* architecture supports several novel features including (i) Virtual Personal Device Servers (VPDS) for local hosting the middleware within the building, (ii) Scripting framework within the middleware for providing rich support for developing and automating energy management applications, and (iii) A rule-based *fine-grained access control* mechanism enables sharing of sensor data and actuation control with other users. Validation of the developed system was done using multiple deployments, from residential to commercial buildings, spread across India and USA.

REFERENCES

- [1] M. Evans, B. Shui, and S. Somasundaram, "Country report on building energy codes in india," in *Pacific Northwest National Laboratory, PNNL-17925*, April 2009.
- [2] AEO, "US Energy Information Administration," *AEO2011: Annual Energy Outlook*, April 2011.
- [3] A. Krioukov, G. Fierro, N. Kitaev, and D. Culler, "Building application stack (bas)," in *Proc. of BuildSys*. ACM, 2012, pp. 72–79.
- [4] A. Molina-Markham, P. Shenoy, K. Fu, E. Cecchet, and D. Irwin, "Private memoirs of a smart meter," in *Proceedings of the 2nd ACM workshop on embedded sensing systems for energy-efficiency in building*. ACM, 2010, pp. 61–66.
- [5] R. P. Singh, S. Keshav, and T. Brecht, "A cloud-based consumer-centric architecture for energy data analytics," in *Proceedings of the fourth international conference on Future energy systems*. ACM, 2013, pp. 63–74.
- [6] P. Arjunan, N. Batra, H. Choi, A. Singh, P. Singh, and M. B. Srivastava, "Sensoract: a privacy and security aware federated middleware for building management," in *Proceedings of the Fourth ACM Workshop on Embedded Sensing Systems for Energy-Efficiency in Buildings*. ACM, 2012, pp. 80–87.
- [7] S. Dawson-Haggerty, X. Jiang, G. Tolle, J. Ortiz, and D. Culler, "smap: a simple measurement and actuation profile for physical information," in *Proc. of SenSys*. ACM, 2010, pp. 197–210.
- [8] H. Choi, S. Chakraborty, Z. M. Charbiwala, and M. B. Srivastava, "Sensorsafe: a framework for privacy-preserving management of personal sensory information," in *Secure Data Management*. Springer, 2011, pp. 85–100.
- [9] W. I. Grosky, A. Kansal, S. Nath, J. Liu, and F. Zhao, "Senseweb: An infrastructure for shared sensing," *Multimedia, IEEE*, vol. 14, no. 4, pp. 8–13, 2007.
- [10] X. Chu, B. Durnota, R. Buyya *et al.*, "Open sensor web architecture: Core services," in *Intelligent Sensing and Information Processing, 2006. ICISIP 2006. Fourth International Conference on*. IEEE, 2006, pp. 98–103.
- [11] K. Aberer, M. Hauswirth, and A. Salehi, "Global sensor networks," *EPFL, Lausanne, Tech. Rep*, 2006.
- [12] R. S. Brewer and P. M. Johnson, "Wattdepot: An open source software ecosystem for enterprise-scale energy data collection, storage, analysis, and visualization," in *Smart Grid Communications (SmartGridComm), 2010 First IEEE International Conference on*. IEEE, 2010, pp. 91–95.
- [13] X. Jiang, "A High-Fidelity Energy Monitoring and Feedback Architecture for Reducing Electrical Consumption in Buildings," *PhD dissertation*, UC Berkeley 2010.
- [14] C. Dixon, R. Mahajan, S. Agarwal, A. Brush, B. Lee, S. Saroiu, and V. Bahl, "An operating system for the home," in *Proc. of NSDI*. ACM, 2012.
- [15] A. Rowe, M. E. Berges, G. Bhatia, E. Goldman, R. Rajkumar, J. H. Garrett, J. M. Moura, and L. Soibelman, "Sensor andrew: Large-scale campus-wide sensing and actuation," *IBM Journal of Research and Development*, vol. 55, no. 1.2, pp. 6–1, 2011.
- [16] Y. Agarwal, R. Gupta, D. Komaki, and T. Weng, "Buildingdepot: an extensible and distributed architecture for building data storage, access and sharing," in *Proceedings of the Fourth ACM Workshop on Embedded Sensing Systems for Energy-Efficiency in Buildings*. ACM, 2012, pp. 64–71.
- [17] S. Dawson-Haggerty, A. Krioukov, J. Taneja, S. Karandikar, G. Fierro, N. Kitaev, and D. Culler, "Boss: Building operating system services," in *Proc. of NSDI*, 2013.

¹²<http://www.trane.com>, <http://www.johnsoncontrols.com>

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60

SensorAct: A Privacy and Security Aware Federated Middleware for Building Management

Pandarasamy Arjunan[†], Nipun Batra[†], Haksoo Choi[§], Amarjeet Singh[†],
Pushpendra Singh[†], Mani B. Srivastava[§]

[†]Indraprastha Institute of Information Technology
Delhi, India

[§]University of California
Los Angeles, United States

{pandarasamya,nipunb,amarjeet,psingh}@iitd.ac.in haksoo@cs.ucla.edu, mbs@ucla.edu

Abstract

The archaic centralized software systems, currently used to manage buildings, make it hard to incorporate advances in sensing technology and user-level applications, and present hurdles for experimental validation of open research in building information technology. Motivated by this, we — a transnational collaboration of researchers engaged in development and deployment of technologies for sustainable buildings — have developed *SensorAct*, an open-source federated middleware incorporating features targeting three specific requirements: (i) Accommodating a richer ecosystem of sensors, actuators, and higher level third-party applications (ii) Participatory engagement of stakeholders other than the facilities department, such as occupants, in setting policies for management of sensor data and control of electrical systems, without compromising on the overall privacy and safety, and (iii) Flexible interfacing and information exchange with systems external to a building, such as communication networks, transportation system, electrical grid, and other buildings, for better management, by exploiting the teleconnections that exist across them. *SensorAct* is designed to scale from small homes to network of buildings, making it suitable not only for production use but to also seed a global-scale network of building testbeds with appropriately constrained and policed access. This paper describes *SensorAct*'s architecture, current implementation, and preliminary performance results.

Categories and Subject Descriptors

H.4 [Information Systems Applications]: Miscellaneous; D.2.11 [Software Engineering]: Software Architectures

General Terms

Design, Architecture, Deployment

Keywords

Middleware, Building management, Sensors, Actuators

1 Introduction

As one of the largest consumers of overall energy, buildings have emerged as attractive targets for using information and communications technologies to advance large scale sustainability goals. Computation technologies promise enabling intelligent sensing, learning, prediction, control, and actuation to be deeply embedded in the fabric of building. Such close integration will result in meeting performance goals with the smallest energy footprint while being responsive to changing usage pattern, external conditions and occupant needs. As a result there is now tremendous research and entrepreneurial activity, as exemplified by the *ACM BuildSys* itself, targeting buildings as an exciting substrate for novel computational methods and technologies that measure, model, understand, and optimize the design and operation of these complex human-cyber-physical systems.

However, these efforts in building-scale energy management are fraught with deployment challenges, many of which are rooted in the inflexible, isolated, slow, and archaic software that forms the *middleware* in the current building management systems. Designed for centralized facilities management departments, currently available middleware are poorly suited to accommodate the needs of emerging research and production systems. Emerging needs include (i) accommodating a richer ecosystem of hard and soft sensors, actuators, and higher level third-party applications, thus helping unleash innovation similar to the one observed in mobile phone segment (ii) participatory engagement of stakeholders other than the facilities department, such as occupants, in setting policies for control of electrical systems and sharing and management of sensor data, without compromising on the overall operational security and integrity, and (iii) flexible interfacing and information exchange with systems external to a building, such as communication networks, transportation system, electrical grid, water network, and other buildings for better management by exploiting the teleconnections that exist across them.

In order to nurture future information technologies for building management and create a robust research ecosystem, it is also important that middleware also provide mechanisms for experimental and research systems to interface

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
BuildSys'12, November 6, 2012, Toronto, ON, Canada.
Copyright © 2012 ACM 978-1-4503-1170-0 ...\$10.00

with a building for sensor information gathering and sub-system control in a suitably limited and controlled fashion taking into account safety, security and privacy considerations of the occupants. Such an ability of the middleware to be research-friendly is important since the complex physical envelope and social milieu that buildings are embedded in, makes them hard to model or to replicate in isolated testbeds, and controlled evaluation of new technologies in operational buildings is essential to scale and transition them into practice. The preceding observations are informed by our own first hand experiences in research involving - development of new sensing and actuation technologies relating to energy and water sustainability; their deployment in diverse testbed (single-family home, student apartments, student dormitories, offices, and dry laboratories) across two cities in two different countries with quite different climatic, economic, legal, social, and cultural settings; and, sharing of testbed access and data across the two different institutions involved.

This paper presents *SensorAct*, a middleware for building management that incorporates several interesting capabilities to help meet the requirements discussed above. *SensorAct* has a twofold vision. First, it seeks to be a platform for production use, providing the thin waist of an hourglass stack for building information technology systems. It enables a diversity of sensors/actuators and applications to sit below and above it respectively, and a participatory approach to building management that engages various stakeholders with overlapping concerns. Second, by enabling a web of interconnected installations, it seeks to seed a global-scale network of building testbeds with suitably constrained and policed access (to meet privacy and safety concerns of occupants, facilities management, and Institutional Review Board requirements for human subjects research) to sensor and actuator resources enabling validation, comparison, and generalization of research. The key features of *SensorAct*, described in depth later in the paper, are:

A tiered and distributed architecture consisting of *Virtual Personal Device Servers* for local storage, access control and device management, and a federation of *Brokers* as a distributed registry of users and device servers resulting in an architecture that combines global access with local control. Local device servers not only permit scaling to higher rate sensing and actuation, but also make it easier to meet safety and privacy concerns.

Powerful sensor and actuator guard rules that enable a firewall using which device owners and occupants of a space may exercise fine-grained control over sensory data management and control of building subsystems.

Lightweight tasking framework that enable end-applications to inject one-shot and persistent event-triggered scripts into the middleware to perform rich forms of sensor querying, sensor processing and fusion, actuator control, and notifications.

2 Related Work

Sensing and control in buildings is addressed at different levels by several systems, that can be broadly classified into four categories - Commercial Building Management Systems (BMS), Home Automation Systems, third party Data

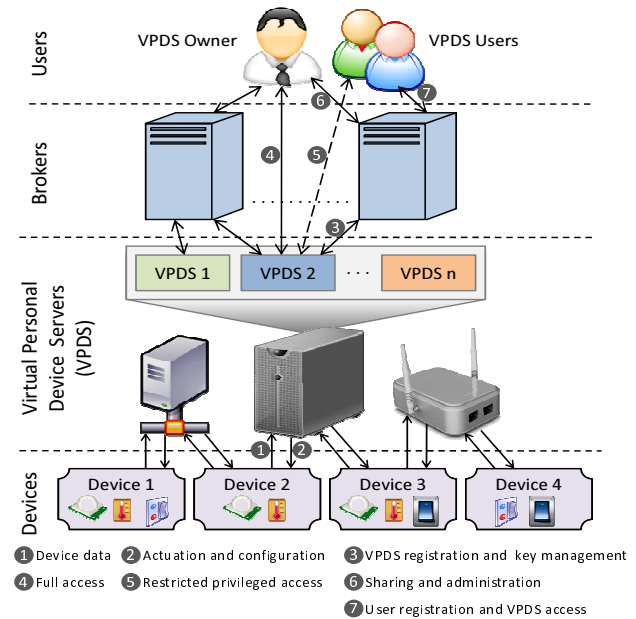


Figure 1: Tiered architecture of *SensorAct* System

Aggregation Services, and Academic Research Systems. We briefly describe each one of these together with related work in privacy-enabling systems for sensor data collections.

Commercial BMS: have emerged as an end-to-end system for management of several building operations e.g. elevator control, fire response, and Heating Ventilation and Air Conditioning (HVAC) control. Of particular reference to our work is HVAC control. Several aspects differentiate the *SensorAct* system from BMS¹ enabled HVAC systems. Software architecture of these systems is targeted towards a centralized control with limited conflict resolution when multiple entities execute control over the same hardware controllers, thus restricting engagement of end user occupant. Further, these software systems expose limited hardware controller capabilities, allowing for control operations that are hard coded in the closed, vendor provided software system. While it is technically feasible for external devices and third-party applications to exchange sense and control information with the BMS systems using open protocols such as BACNet and OPC (Openness, Productivity and Collaboration) standard tunnels, complexity of these protocols and limited exposure of hardware controller capabilities restrict the support for multitude of new sensors (e.g. communicating over ZigBee and Z-Wave), actuators and third-party applications. Their expensive cost structure and stovepipe system architecture make BMS inaccessible to individual homeowners and constrained for data and control sharing across multiple organizations.

Home Automation Systems: A separate segment of industry has catered to the requirement of individual homeowners providing integrated hardware and software systems²

¹Examples of commercially available BMS include

<http://www.trane.com>, <http://www.johnsoncontrols.com>, <http://www.buildingtechnologies.siemens.com>

²<http://micasaverde.com>, <http://www.homeseeer.com>, <http://www.homeseeer.com>

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60

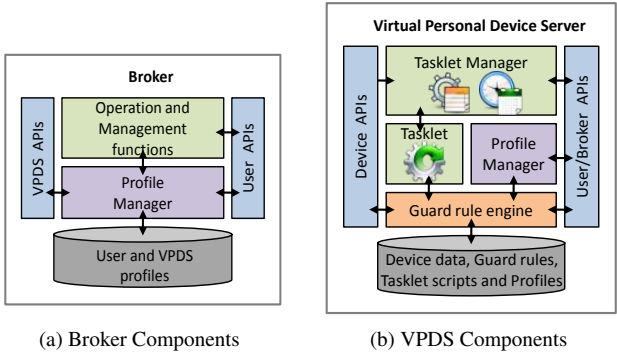


Figure 2: Components of Broker and VPDS

and systems primarily providing software control with flexibility to integrate with diverse hardware systems³. These systems are primarily targeted towards security of the homes and comfort of its occupants rather than energy conservation. Visualization and script support in many of these systems provide users with increased engagement. While many of these systems support local data storage, there is limited support for sharing sense and control across multiple home deployments of these systems.

Data Aggregation Services: Several web-services have been introduced recently that allow data aggregation and visualization⁴. However, these services only provide limited support for actuation, primarily catering to messaging triggers. While RESTful interface allow these services to aggregate data from geographically separated sensors, centralized data storage architecture results in these services imposing significant sensor data rate limits. Even though their centralized architectures allow easy sharing of data across multiple users, these systems provide limited or no support for privacy and security of the end user thus restricting the data sharing to “all-or-nothing” model, rather than controlling the information released as function of context and sensor value.

Research systems: Several sensor based research systems, such as SenseWeb [11], SensorWeb [5] and GSN [1], have been developed and deployed. IBM’s InfoSphere⁵ platform provides a set of commercial software tools for data integration, information management and real time stream data analytics. But these tools were designed for enterprises and provides no support for data and control sharing capabilities.

Privacy-aware mobile data collection: Primary inspiration from the literature in privacy is that the users should be given appropriate control over sharing of their data and actuation of their devices [12]. This philosophy has been earlier adopted in other storage systems, in mobile data collection [3, 4, 6]. These systems propose personal data repositories of sensory data, using virtualization as a way to eliminate exposing data to third parties as would be the case with data aggregation services. Building on this inspiration, SensorAct system separates “hosting” from “sharing”,

³<http://www.perceptiveautomation.com>, <http://www.eragy.com>
⁴<http://www.eragy.com/MyEragy/>, <https://cosm.com>, <http://www.nimbits.com>, <https://www.thingspeak.com>, <https://code.google.com/p/wattdepot/> [2]
⁵<http://www.ibm.com/software/data/infosphere>

unlike data aggregators wherein a central data storage does not allow fine-grained control over release of information or control of actuators. Privacy notions in SensorAct system architecture differ from these earlier proposed systems to additionally allow for security of actuators and data transformation mechanisms before releasing the data.

3 SensorAct Architecture

SensorAct is based upon tiered architecture as shown in Figure 1. The main components of SensorAct are Virtual Personal Device Server (VPDS) and Broker, interacting with devices at the lower layer and users and third-party applications at the higher layer.

3.1 Devices

A device can consists of hard and soft sensors and actuators. Sensory interface allows users to read the current state e.g. PIR sensor specifying motion. Actuators allow users to change the state of an appliance e.g. switching on the air-conditioner as well as setting the parameters for a sensory interface e.g. setting the sampling rate of a sensor. A device consisting of an actuator should specify an IP address for its accessibility. Each device is owned by a single user. Besides, the physical sensors and actuators, SensorAct also supports:

Computed sensors: are abstractions that can be calculated based on a pre-defined mathematical function applied on a single or multiple sensor values. For example, a computed sensor can specify average temperature in a room over 10 minutes (calculated using temperature values observed every second) and a computed sensor InMeeting can be set based on multiple occupants in the room (using a combination of motion sensor and microphone sensor).

Grouped actuators: are abstractions that can control multiple actuator e.g. power off the room may result in turning off air-conditioner as well as lights. These are similar to “scenes” as specified in several commercial home automation systems.

3.2 Virtual Personal Device Servers (VPDS)

VPDS consists of a database, a Tasklet manager, a Guard rule engine, a profile manager and APIs for devices, users, and brokers to interact with the VPDS, as shown in Figure 2b. A single physical server may host multiple VPDS, however virtualization is used to ensure the isolation of VPDS and privacy of the data within it. A single user owns a particular VPDS; though, the owner may allow other users to have controlled access to the sensors and actuators associated with the VPDS.

A VPDS Owner (VO) can manage multiple devices owned by him through the profile manager. A device profile consists of a set of attributes such as its name, IP address, a collection of sensor and/or actuator profiles, number of channels, data types and units, location, placement, and exposure. Additionally, a set of tags as < key,value > pairs may be associated to let the user flexibly annotate additional details of the device. Device profile is represented using hierarchical model to name and identify devices, sensors, actuators, channels, and their readings uniquely e.g. building : floor : room : device : id : [sensor|actuator] : id : channel : data. A device search or data query can consists of a set of these

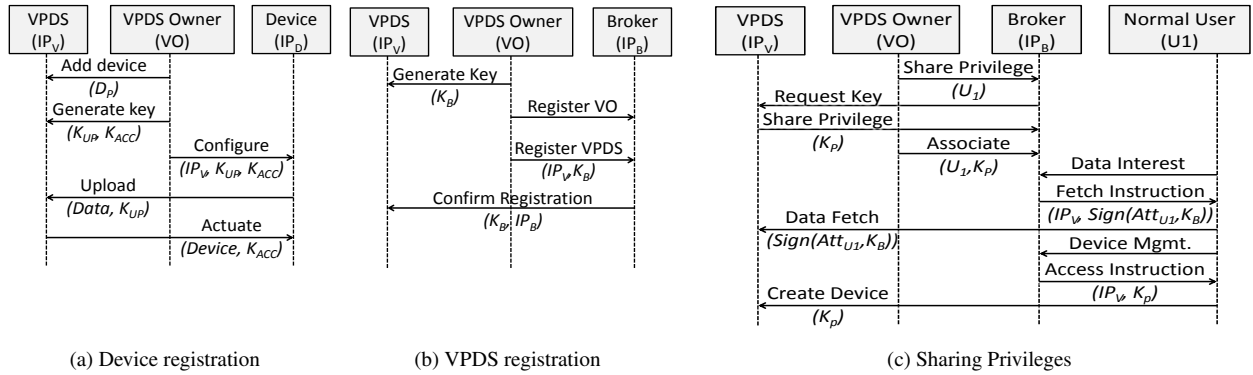


Figure 3: Message exchange between VPDS Owner, VPDS, Broker and other users for different operations in the SensorAct system

attributes. VO also generates an upload key and an access key at the VPDS to be used in the devices for authorizing the data upload into the VPDS and actuation access by the VPDS respectively. Both the keys, together with the VPDS address are then configured manually in the device to register the device with the VPDS, as shown in Figure 3a. A device can also be associated with more than one VPDS. Once a device is registered with a VPDS, it can upload the data or take actuation commands from the corresponding VPDS.

A schema-less, key-value based database is used to store the sensor data from devices, allowing for efficient storage and querying of unstructured time-series data. The database stores WaveSegs (see Section 3.6) and can only be accessed via the *guard rule engine* in the VPDS. The guard rule engine is a set of guard rules (see Section 3.5) which are used by VO to restrict access to the sensory data and actuation, while sharing it with other users. *Tasklets* (see Section 3.7) are lightweight scripts used to perform rich forms of one-shot and persistent event-triggered actions including sensor querying, sensor processing and fusion, actuator control, and notifications. The triggers are in terms of complex events specified over the real-time clock and real-time sensor measurement. A task scheduler is used to execute the tasks.

The VPDS abstraction not only permits flexible provisioning of hardware resources but also allows diverse deployment scenarios ranging from VPDS hosted on a local machine for high-rate low-latency sensing and control to VPDS running on cloud-based hosting services such as Amazon EC with lower-rate sensing and control.

3.3 Broker

In SensorAct, a trusted broker contains a registry of users, a registry of VPDSs, and acts as a mediator to help clients establish a connection with VPDSs. Figure 3b illustrates the process for registering a VPDS to a broker. First, VO generates a key (K_B) on the VPDS. Secret key (K_B) together with VPDS attributes (e.g. IP address) are then used to register it with a broker. Upon registration, the broker contacts the VPDS with K_B and correspondingly the broker is added in the list of trusted brokers at the VPDS. A broker also allows a VO to share administrative privileges and data and control access, for his/her VPDS, with other users on the broker. A trusted broker is authorized to perform key management for the VO to share administrative privileges with other users.

Broker also signs user attributes with a secret key for data or actuation access to the VPDS that is directly performed between the user and the VPDS. More details on privilege sharing is discussed in Section 3.5. In the future, we also plan to support federation of multiple brokers.

3.4 Users

In SensorAct, a user may have two roles: i) Owner of his/her own devices and correspondingly associated VPDS ii) User with data and control access as per the privileges assigned by the owner other devices. A single user could be the owner of his/her own VPDS and user for some other VPDS. As an owner, a user may grant controlled access to other users thus letting them access sensor data from the user devices or to even control them in a carefully constrained fashion. Users may use diverse end-applications to consume sensor data and control actuators in a device. These applications may provide users with more convenient interface to the underlying APIs and tasklet mechanism. SensorAct comes with a browser-based cloud-hosted application to let users interact with brokers and VPDS, but alternative end-applications can easily be implemented in diverse programming languages.

3.5 Sharing privileges

Two sets of privileges are associated with each VPDS - administrative privilege for guard rule, task and device management; and data and control access. When a VO wants to share some administrative privileges, the trusted broker (where the VPDS is registered) contacts the corresponding VPDS and is provided with a secret key (K_P), as shown in Figure 3c. Each key provided by the VPDS has associated administrative privileges to perform one or more of the guard rule, task and device management operations. A single key may be shared across multiple users to provide them with same group privileges. When a user wants to perform administrative action, s/he requests for the VPDS address and privilege key from the broker and thereafter directly contacts the VPDS to perform the required action. Corresponding API request by the user at VPDS is authenticated using K_P .

API requests for data and control access are authenticated using user credentials, signed by the broker, and verified by a set of “guard rules”. Each VPDS internally maintains a set of guard rules, created by its owner and by other users to whom the owner has given the privilege to create guard

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60

rules. These guard rules are intended to meet privacy and safety concerns associated with sharing of one’s sensors and actuators. For example, the expressive guard rules permit policies such as restricting access to a sensor by a user or group of users as a function of time of day, sensor location, and current measured value. This is considerably more powerful than the all-or-nothing access to a sensors data that current user authentication based data aggregation services and other research systems provide. Moreover, the guard rules also police access to the actuators, and ensure that a user may not issues unsafe commands or sequence of commands to an actuator. For example, facilities management may limit the range within which a user may set an actuator, or how frequently a user may change actuator settings.

By default, owner has all the privileges while other users have no access unless permitted by the guard rules. Hierarchy of users exercise control over shared devices using nested rules wherein conflicts are avoided using priorities associated with each rule. For the convenience of the VPDS owner, SensorAct system provides a set of template rules that can be filled in with required parameters to create concrete rules. Users can create their own template rules or use the template rules provided by the system. Further flexibility in guard rule definition is provided through support for globally defined macros and in-built functions for data obfuscation. Users can define their own groups of users using macros or specify multiple users by using regular expressions in user condition.

Figure 4 illustrates sample guard rules and associated architectural features. Figure 4a shows a guard rule that allows all users in SensorAct.edu domain to access data collected during work time only. However, the rule obfuscates the data by adding Gaussian noise (provided as an internal function by SensorAct system) with mean 70 and variance 15. *WORK_TIME* in condition field is a macro defined through corresponding API with JSON object shown in Figure 4b. To define repeat time, Unix Cron time expression is used to specify 9am to 6pm during weekdays. Figure 4c is an example of a template rule. The rule allows a user to change an actuators value within a certain range, which is parametrized. The concrete instantiation of the template rule is shown in Figure 4d, which specifies the parameter values as 60 and 100.

3.6 Sensory Information Representation

An important aspect of SensorAct is that it needs to handle large volumes of data generated by a multitude of sensors. Storing the time series of sensor data as individual tuples is inefficient both in terms of storage size and querying time. In order to provide an abstraction of sensors that is generic, compact, efficient, and scalable to diverse sensing modalities and sampling policies, SensorAct represents the continuous sensor data streams using *WaveSegs*, an abstraction for sensor waveforms used in [4] and in turn inspired by *SigSeg* used in MIT’s WaveScript system [9]. *WaveSegs* refers to non-overlapping windows of the sensor waveform, and are the atomic units from which a sensor waveform in SensorAct is composed. Sensors send measurements to SensorAct as *WaveSegs*, and SensorAct’s storage is also organized in terms of *WaveSegs* instead of individual samples.

```
{
  "NAME": "AddNoiseRule",
  "TARGET_OPERATION": "READ",
  "DESCRIPTION": "Add gaussian noise to data on
                  work time for all users at SensorAct.edu."
  "PRIORITY": 1,
  "CONDITION": "USER.email == *@SensorAct.edu
                && TIME == WORK_TIME",
  "ACTION": "AddGaussianNoise(70, 15)"
}
```

(a) Normal guard rule

```
{ "MACRO_NAME": "WORK_TIME",
  "MACRO_VALUE": "[ *9-18 * * 1-5 ]" }
```

(b) Guard rule macro

```
{
  "NAME": "ValueRangeTemplate",
  "DESCRIPTION": "Allow changing actuator
                  values within specified range."
  "TARGET_OPERATION": "WRITE",
  "CONDITION": "VALUE >= #PARAM_MIN_VALUE
                && VALUE <= #PARAM_MAX_VALUE",
  "ACTION": "ALLOW"
}
```

(c) Template guard rule

```
{
  "NAME": "ValueRangeRule",
  "TEMPLATE_RULE_NAME": "ValueRangeTemplate",
  "DESCRIPTION": "Allow changing actuator values
                  within 60 to 100."
  "PRIORITY": 1,
  "PARAMETERS": { "PARAM_MIN_VALUE": 60
                  "PARAM_MAX_VALUE": 100 }
}
```

(d) Concrete rule instantiated from the template rule

Figure 4: Example guard rules

Within a WaveSeg the sampling policy is fixed, with support for both isochronous (periodic or uniform) sampling and asynchronous (aperiodic or non-uniform or adaptive) sampling. In the former case, SensorAct leverages isochronicity for compactness of representation by foregoing explicit annotation of samples with timestamps and instead associating a sampling period with a WaveSeg, as is also illustrated in the example below. Each WaveSeg contains self explanatory metadata about the sensor readings such as location, device name, sensor name, sensor id, sampling interval and start time of the readings to enable rich data querying capabilities. Additionally, each channel within a sensor is separately specified with channel name, units for the readings and an array of float values specifying the sensor readings. Additional metadata information, such as location, can also be easily added to this description. Figure 5 illustrates the JSON representation of a WaveSeg, as used by a sensor device to upload data. While WaveSegs significantly improve upon per-sample storage, the number of WaveSegs stored in SensorAct’s database nevertheless directly affects query processing performance. To further optimize performance, SensorAct opportunistically merges WaveSegs as they are uploaded by a sensor.

3.7 Lightweight Tasking Framework

The tasking framework of SensorAct supports *tasklet*, light-weight non-blocking scripts, to perform sense-and-control operations within the system. Tasklets provide a generic way to perform several system operations such as querying current and historical sensor data, real time actuation, complex event management and processing, and customized notifications and alerts. The tasklet framework follows a simple programming model using the following

```

{
  "DEVICE_NAME": "Office_Flyport",
  "SENSOR_NAME": "MultiSensor",
  "SENSOR_ID": 1,
  "SAMPLING_INTERVAL": 1,
  "EPOCH_TIME": 1344147449,
  "CHANNELS": [
    {
      "NAME": "Temperature",
      "UNIT": "Celsius",
      "READINGS": [28.1,28.2,28.6,28.5,28.2,28.6,28.5,28.7]
    },
    (...)
  ]
}

```

Figure 5: JSON representation of a sample WaveSeg

primitives:

- **Input** - List of system devices and timers that act as triggers for the corresponding action specified in the tasklet. This is inspired by sensitivity list used in VHDL programming language.
- **When** - Specifies a complex boolean operation using AND, OR primitives as a combination of triggers specified as inputs.
- **Execute** - Specifies the corresponding action to be performed when the condition in the “When” primitive holds true.

Figure 6a shows an example tasklet. “Input” consists of a time based trigger that becomes active every 2 minutes between 10AM and 6PM everyday. “When” primitive specifies a simple condition which holds true every time the trigger is active. “Execute” primitive specifies a script *monitor_ac.lua* that is executed whenever “When” primitive is true, in this case every 2 minutes between 10 AM and 6 PM everyday. Additionally, parameter list using *PARAMS* primitive can be used for parameterized action scripts. Using various inputs, proposed tasklet model can support diverse operations as follows:

One-shot operations e.g. querying current status of a device are supported by specifying null triggering condition resulting in the action script getting executed immediately and only once.

Periodic operations are supported by specifying periodic timing conditions as triggers, as is also illustrated in the example discussed in Figure 6a.

Event driven operations e.g. real time monitoring and interactions with devices, are supported using only devices as triggers, resulting in the action script getting executed whenever a change in the device value is observed.

Periodic and Event based operations include fine-grained monitoring and control, are supported using a combination of devices and timers as triggers. In this case, the action script will be executed whenever a change in the state of the device is observed and/or the timing condition is satisfied (depending upon the boolean expression in the “When” primitive).

Action scripts are independent light-weight and non-blocking programs, very similar to interrupt service routines, that performs read and write operations on devices and computes simple calculations and branching using the read data.

```

{
  "NAME": "Monitor_AC",
  "PARAMS": {
    "T1": "Mickey:Room1:Tempr:1",
    "A1": "Mickey:Room1:AC:1",
    "MINS": 5,
    "LIMIT": 30
  }
  "INPUT": {
    "TIMER1": "[0 0/2 10-18 * *]"
  }
  "WHEN": "TIMER1",
  "EXECUTE": "[monitor_ac.lua]"
}

```

(a) Example tasklet format

```

-- monitor_ac.lua
-- Reads sensor readings and
-- actuate appliance accordingly
-- epoc MINS minutes before
epocNmin = os.time() - (60*MINS)
-- read MINS minutes avg value
avgTr = VPDS:readAvg(T1,epocNmin)
-- Check and turn ON
if avgTr > LIMIT then
  VPDS:write(A1,VPDS:TURNON)
end

```

(b) Tasklet script (Lua)

Figure 6: Tasklet example: Every 2 minutes from 10AM to 6PM every day, read the past 5 minutes average temperature of room1 and turn on the Air-Conditioner in the same room if the average temperature is above 30

Any read/write operation to a device made by the tasklet pass through the guard rule engine, explained in Section 3.5. As a result, only those read/write operations that are authorized by the device owners will be allowed. The tasklets will inherit the permissions of the user who is executing it.

A *Tasklet Manager* receives the tasklet execution requests and executes a tasklet when its corresponding triggering conditions are met. Each tasklet runs with its own independent execution context which is monitored by the tasklet manager. As a result, no data exchange is supported across tasklets. Once a tasklet is submitted, the tasklet manager validates the task execution request and registers a trigger to the system that monitors for the events (timer and device data) based upon its triggering condition. A handle is assigned to the tasklet and is returned to the user for future reference that may include knowing the status or canceling the tasklet. Tasklet manager also keeps track of any data produced by the tasklet, particularly the data produces by periodic tasklets. This data is accessible by the user executing the tasklet using the handle of the tasklet.

4 Implementation

Implementation of the architecture, as proposed in Section 3, is an ongoing activity. We used Java and Play framework⁶ to implement multiple components of VPDS and Broker and schema-less MongoDB as database. We released the system implementation code in open source⁷ on web hosted code repository and welcome community contribution and usage. Table 1 lists the SensorAct APIs currently supported in each of the system component. We also created a user interface application, using several Web 2.0 technologies, for VPDS owner to interact with the VPDS. Figure 7 shows the snapshot of a sample user interface web application. The current implementation of guard rule supports allow and deny actions on read and write operations with basic conditions. Arbitrary boolean expression are supported for conditions *USER.email*, *LOCATION_TAG* (exact match), *TIME* (unix epoch time) and *VALUE*.

The *Tasklet manager* uses Quartz⁸, an industrial-strength task scheduler that scales well to large number of tasklets besides interfacing well with the underlying Java-based implementation of SensorAct. The scheduler supports both persistent and one-time tasks. In the current implementation, a

⁶<http://www.playframework.org>

⁷<https://github.com/iiitd-ucla-pc3>

⁸<http://quartz-scheduler.org>

Component	VPDS APIs
User	/user/{register login list}
Key	/key/{generate delete list enable disable}
Device	/device/{add delete get list search share}
	/device/template/{add delete get list}
Guardrule	/guardrule/{add delete get list}
	/guardrule/association{add delete get list}
Tasklet	/tasklet/{add delete get list}
	/tasklet/{execute cancel status}
Data	/data/{upload/wavesegment query}
Component	Broker APIs
User	/user/{register login}
VPDS	/vpds/{register remove}

Table 1: Supported SensorAct APIs for different system components

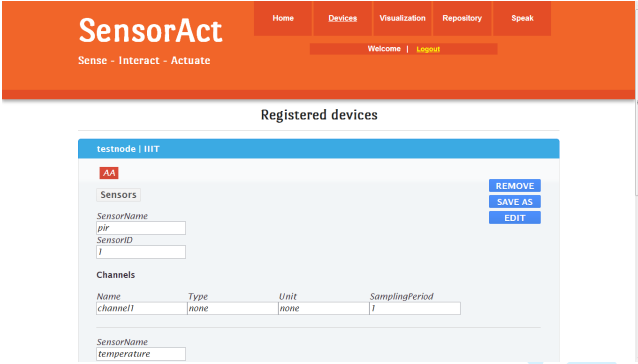


Figure 7: Snapshot of sample user interface web application

background job is initiated on receiving a tasklet to monitor the corresponding triggers. Whenever the registered trigger is raised, corresponding tasklet will be executed in a separate worker thread, managed by tasklet manager.

We use Lua⁹, a lightweight scripting language, to express tasklet script. Lua interpreters are known for small memory footprint and fast execution, and Lua itself lends well to domain-specific extensions. We use Java Scripting API framework to execute lua scripts through jnlua¹⁰, a Lua script engine that integrates Lua into Java and supports language bindings. Figure 6 illustrates a sample tasklet execution request and the corresponding Lua script, demonstrating the flexibility and compactness of the SensorAct’s scripting framework. Users can use the currently supported interface to write their own tasklets. We also plan to provide support for templates and graphical user interface to create common tasklets and guard rules.

5 Evaluation

The complete evaluation of SensorAct requires extensive deployment of hardware systems and a detailed user study regarding the the usability and utility of the system features. As this first work primarily focus upon the system architecture, detailed evaluation is outside the scope of this paper. In this section, we explain about our ongoing deployment plan and the performance evaluation of SensorAct by emulating the proposed deployment scenario.

5.1 Deployment plan

Proposed SensorAct will be deployed in diverse environments by multiple collaborators, including two academic

partners - IIIT Delhi and UCLA and an industrial research laboratory - IBM Research, India. Our largest deployment in IIIT Delhi campus involves sensing infrastructure across 400 rooms in student dorms with hardware support for up to 10 channels of sensory data per room, monitoring various energy, ambient, and occupancy variables at 1 Hz. We further plan to extend the sensing infrastructure to monitor faculty offices and interact with commercial Building Management System (put in place for HVAC Control). Sensing and control for all the infrastructure will be done using proposed SensorAct system. Another academic deployment at UCLA involves a 1200 sq. ft. laboratory space with 30 electrical channels each and 10 event driven sensors for motion, door, and light. Proposed system deployment for Softgreen [13] testbed at IBM Research India involves more than 75 sensing nodes collecting temperature, motion and various soft sensors to infer employee occupancy information.

For each of the ongoing deployments, a multitude of sensing platforms are being used for monitoring diverse set of parameters. These platforms include Z-Wave based sensors and plug computers, panel monitors, water flow sensors and Ethernet/Wi-Fi based microcontroller platforms. We were able to easily collect data from all these diverse sensors into the SensorAct system with minimal effort. Experience with these diverse deployment experience shows the general applicability of the SensorAct system architecture.

5.2 System Performance

We experimented the performance and scalability of SensorAct by emulating our largest deployment scenario. In this setup, 400 identical processes running in parallel upload sensor readings, each sending 100 sensor measurements (10 sensing channels at 1 Hz) every 10 seconds in the form of a WaveSegs. All the data upload processes were hosted on a laptop connected over Intranet with another laptop (2.3 Ghz Intel Core i7 processor, 8 GB RAM, 5400 RPM Hard Disk) hosting SensorAct. We disabled the Guard Rule engine to test the baseline performance. We conducted this data upload experiment for 40 minutes. We verified that all the WaveSegs were aggregated. The average and median CPU usage was 14.6% and 7% and average and median RAM usage was 3.7% and 3.2% respectively. Results clearly indicate the scalability of the SensorAct system for large scale data aggregation from low frequency sensors.

The overhead of guard rule engine is based upon the number of rules, target operation (read or write) and number and size of WaveSegs. Tasklet performance is based upon the complexity of the script, size of object binding between Lua and Java, and the number of read/write requests that goes through the guard rule engine. To emulate a common read operation by a normal user, we setup three guard rules - one checking for the user email (at the WaveSeg level) and the remaining two checking for the timestamps and the sensor values (to be done for each reading in the WaveSeg). We emulated the read operation, querying for 15 minutes of 1 Hz sensor data as a nil triggered tasklet that reads 90 WaveSegs, each containing 10 data values.

In the first experiment, we scheduled to execute the tasklet once in every 5 seconds for a total period of 30 minutes. We observed that, on an average, guard rule engine takes 3356ms

⁹<http://www.lua.org>

¹⁰<http://code.google.com/p/jnlua>

(including 10ms for database read) to process guard rules for each tasklet and each tasklet execution instance takes 3363ms (including guard rule processing time). Since this operation will be infrequent, current processing time will work fine. However, we will be working on improving the guard rule processing as proposed in the future work. To emulate the data access by VPDS Owner, we repeated the experiment without the guard rules. In this case, on an average, guard rule engine takes 11ms to process guard rules for each tasklet and each tasklet execution instance takes 11.5ms (including guard processing time). Average CPU usage during these experiments were 68% and 2% respectively on a laptop (2.4GHz Intel Core i5 processor and 2GB RAM).

6 Conclusions and Future Directions

SensorAct system is a result of the needs that we faced in our own transnational collaborative research: an open, flexible, taskable, and scalable information substrate for buildings into which multitude of sensors, actuators, and applications could be integrated; a building management system approach in which the occupants are engaged as participatory stakeholders instead of just passive consumers; and, exchange of data and limited access to testbeds in a fashion that was cognizant of IRB constraints and sensitive to occupants privacy and buildings safety. These experiences motivated the proposed architectural design of SensorAct system.

While the initial implementation is done and in early stages of being incorporated into our deployments, SensorAct itself continues to evolve. Some specific areas of development include seamless and intuitive device management functions (automatic new device registration and removal of obsolete devices), location and context based device search and data query, implementation of federated broker aspect of the architecture, performance optimization of the guard rules via methods such as dynamic code compilation techniques in packet filters used for network firewalls [8, 14], and supplementing guard rules which target behavioral privacy with an anonymizing and aggregating proxy for privacy of identity. Moreover, with the development code tree open source and hosted on a cloud-based hosting and version control service, we hope to engage other researchers as contributors of code to SensorAct and to link their testbeds via SensorAct.

7 Acknowledgments

This work is partially supported through Indo-US PC3 collaborative program, supported by NSF, USA (Grant Number CNS-1143667) and DEITY, India. We also acknowledge

support of IBM Research, India for using SensorAct for their data collection and for partial support of Pandarasamy Arjunan through IBM PhD Fellowship. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the funding agencies.

8 References

- [1] K. Aberer, M. Hauswirth, and A. Salehi. Global Sensor Networks. In *Technical report LSIR-REPORT-2006-001*, 2006.
- [2] R. Brewer and P. Johnson. WattDepot: An Open Source Software Ecosystem for Enterprise-Scale Energy Data Collection, Storage, Analysis, and Visualization. In *First IEEE International Conference on Smart Grid Communications*, SmartGridComm, 2010.
- [3] R. Cáceres, L. Cox, H. Lim, A. Shakimov, and A. Varshavsky. Virtual Individual Servers as Privacy-Preserving Proxies for Mobile Devices. In *Proceedings of the 1st ACM workshop on Networking, systems, and applications for mobile handhelds*, MobiHeld, 2010.
- [4] H. Choi, S. Chakraborty, Z. M. Charbiwala, and M. B. Srivastava. SensorSafe: a Framework for Privacy-Preserving Management of Personal Sensory Information. In *Proceedings of the 8th VLDB international conference on Secure data management*, SDM, 2011.
- [5] X. Chu, T. Kobialka, B. Durnota, and R. Buyya. Open Sensor Web Architecture: Core Services). In *4th International Conference on Intelligent Sensing and Information Processing*, ICISIP, 2006.
- [6] S. Dawson-Haggerty, X. Jiang, G. Tolle, J. Ortiz, and D. Culler. sMAP: a Simple Measurement and Actuation Profile for Physical Information. In *Proceedings of the 8th ACM Conference on Embedded Networked Sensor Systems*, SenSys, 2010.
- [7] C. Dixon, R. Mahajan, S. Agarwal, A. Brush, B. Lee, S. Saroiu, and V. Bahl. An Operating System for the Home. NSDI, USENIX, 2012.
- [8] D. Engler and M. Kaashoek. DPF: Fast, Flexible Message Demultiplexing using Dynamic Code Generation. In *ACM SIGCOMM Computer Communication Review*, volume 26, pages 53–59. ACM, 1996.
- [9] R. Newton, L. Girod, M. Craig, S. Madden, and G. Morrisett. WaveScript: A Case-Study in Applying a Distributed Stream-Processing Language. *system*, 1(2008/1):31, 2008.
- [10] A. Rowe, M. Berges, G. Bhatia, E. Goldman, R. Rajkumar, J. H. Garrett, J. M. F. Moura, and L. Soibelman. Sensor Andrew: Large-scale Campus-wide Sensing and Actuation. *IBM Journal of Research and Development*, 2011.
- [11] A. Santanche, S. Nath, J. Liu, B. Priyantha, and F. Zhao. SenseWeb: Browsing the Physical World in Real Time (Demo Abstract). In *ACM/IEEE Information Processing in Sensor Network*, IPSN 2006.
- [12] K. Shilton, J. Burke, D. Estrin, R. Govindan, M. Hansen, J. Kang, and M. Mun. Designing the Personal Data Stream: Enabling Participatory Privacy in Mobile Personal Sensing. TPRC, 2009.
- [13] L. V. Thanayankizil, S. K. Ghai, D. Chakraborty, and D. P. Seetharam. Softgreen : Towards Energy Management of Green Office Buildings with Soft Sensors. In *Fourth International Conference on Communication Systems and Networks*, COMSNETS, 2012.
- [14] Z. Wu, M. Xie, and H. Wang. Swift: A Fast Dynamic Packet Filter. In *Proceedings of the 5th USENIX Symposium on Networked Systems Design and Implementation*, pages 279–292. USENIX Association, 2008.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60

Preliminary design of SensorAct middleware system was presented earlier in 2012 Workshop on Embedded Systems for Energy Efficiency in Buildings (Buildsys). Following sections/figures/tables are the new inclusions in this paper

- 1. Section I - Introduction
 - a. It is revised entirely specific to bringing IoT aspects of the middleware
 - b. Figure 1 - SensorAct Architecture is revised
- 2. Section III – Scripting Framework
 - a. A separate section about the scripting framework with elaborated details
 - b. A new subsection III.A about Tasklet API functions
 - c. Figure 3 – Scripting framework workflow
- 3. Section IV – Implementation
 - a. Detailed description of the complete implementation of the system including gateways, and several applications (web portal, time and presence based actuation and mobile application)
- 4. Section V – Evaluation
 - a. This section is entirely new in this paper
 - b. Detailed evaluation the system with several deployments and case study energy management applications